

Cash-out Forecast Console

The Field Guide — everything important about the project, in fifteen pages.

Forecast, don't just detect

Honesty-first ML

Predictive Analytics for Cybercrime Cash-out Forecasting.

The fast, portable companion to the Complete Manual.

1 · The problem & the big idea

FIELD GUIDE

When someone is scammed, the fraudster instantly moves the money through a chain of "mule" bank accounts and pulls it out as cash at an ATM — usually within minutes. Once it's cash, it's gone. This system takes a fraud complaint, traces where the money went, and forecasts, for each account still holding stolen funds: **will it be cashed out, when, how, and roughly where.**

"We don't just detect fraud after the fact — we forecast the cash-out, so police can freeze the account in time, and when they can't, we tell them which area to watch."

Why a forecast, not a report

The money reaches an ATM in minutes, but the average complaint is filed ~110 minutes after the fraud. So the winning move is not "catch them at the ATM" — it's **"freeze the account before the ATM."**

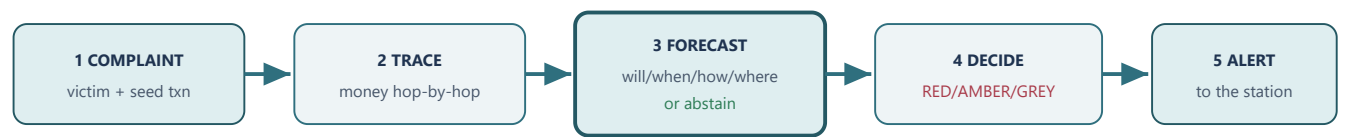
Freezing takes seconds; a dispatch takes tens of minutes. The whole system optimises for a fast, well-justified freeze, with a geographic fallback for when freezing fails.

THE HONEST REALITY

In ~66% of complaints the money was already cashed out before the complaint was even filed. We are useful in the cases where action is still possible — and we use that number to argue for the real fixes: faster reporting and proactive freezing.

2 · How it works — the five steps

FIELD GUIDE



Complaint → trace → forecast → decide → alert. The graph is rebuilt fresh for every prediction.

A complaint names one victim and one disputed "seed" transaction. A time-respecting breadth-first walk (`graphx/trail.py`) reconstructs where the money went, attributing "taint" proportionally through each split and flagging accounts that are **still holding** stolen funds — the freeze targets. For each target, three models forecast the cash-out. Plain rules turn that into an action. An alert routes to the covering police station.

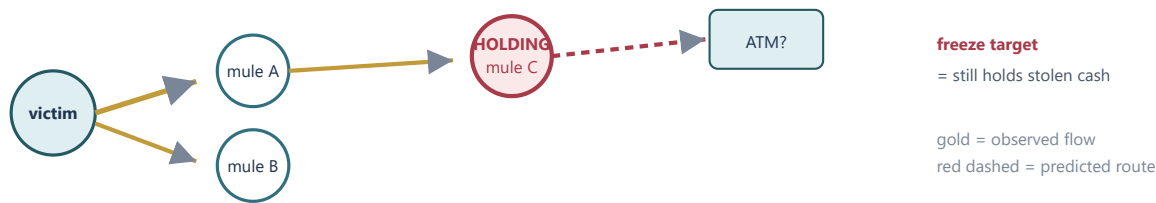
The four I4C deliverables

#	Deliverable	Where
1	Predictive engine (M1/M2/M3)	<code>m1/</code>
2	GIS risk map with filtering	<code>/hotspots</code> + Leaflet map
3	Law-enforcement interface	<code>api/</code> + console
4	Alert & notification system	<code>/alerts/dispatch</code>

3 · The money trail & the graph signal

FIELD GUIDE

Before any model runs, the system reconstructs where the stolen money went. This is done fresh for every prediction — there is no giant precomputed graph, because that would leak the future.



A trail: taint flows victim → mules; an account that received more than it forwarded is "holding" — the thing a freeze can still save.

A time-respecting breadth-first walk follows only edges timestamped before the as-of moment, ignores dust (<5% of the seed), attributes "taint" proportionally through each split, and stops at 4 hops or a cash-out. On top of the trail, nine **graph features** capture where an account sits in the fraud structure — the signal you cannot get from its own transactions alone.

Graph feature	Reads
hop_from_victim	how deep in the laundering chain
in/out_deg_1h	fan-in and fan-out in the last hour
dispersion / flow_concentration	how widely money spreads; how concentrated inflows are
ring_size / ring_prior_cashout_rate	the co-operating ring and its history
n_tainted_predecessors	how many upstream accounts carry stolen money
mule_score	a logistic "how mule-like" over features 1–8

4 · The three models

FIELD GUIDE

Three questions asked about one account that just received stolen money:

M1 — will it be cashed out, and when?

A discrete-time hazard model scored at 10, 20, 30, 45, 60 minutes, stitched into a rising cumulative "hazard curve." From it we read the **action window** (steepest-rise interval) and a severity: imminent / soon / later / unlikely. The 10-minute probability drives the decision.

M2 — how will the money leave?

Four classes: **ATM**, **POS**, **onward_transfer**, **dormant**. The last two stop police rushing an ATM when the money is about to move again or is going nowhere.

M3 — where? (and the part we're proudest of)

M3 ranks candidate ATMs, but only commits to the precision it can defend:

R5 — HIERARCHICAL ABSTENTION

If the top ATM's probability ≥ 0.22 and entropy $\leq 2.60 \rightarrow$ name a **terminal** (top-5 ATMs). Else if the top cell $\geq 0.30 \rightarrow$ name a **~5 km² cell** and record it abstained. Else $\rightarrow$ name only the **district**. The output always says *why* it abstained. The system knows what it doesn't know.

Candidates set the ceiling

M3 only ranks a shortlist of ATMs, built from five sources: the account's own past ATMs (C1), ATMs near its withdrawal centroid (C2) and its KYC PIN (C3), **its ring co-members' ATMs (C4 — the differentiator)**, and the district's top fraud ATMs (C5). Recall@50 = **0.82** — the hard ceiling on location accuracy.

5 · The six honesty rules

FIELD GUIDE

This is what makes the project credible. Six rules keep the science honest, and each is enforced by machinery — not by trust.

Rule	Forbids	Enforced by
R1 as-of	using data from after the prediction moment	every feature reads only <code>ts<as_of</code> ; the leakage test rebuilds a truncated corpus and asserts identical features
R2 label prefix	a label reaching a model	<code>drop_leaky_columns</code> strips all <code>label_*</code>
R3 no live location	a suspect's real-time GPS	only KYC centroid + historical footprint; true home coords never shipped
R4 honest metrics	suspiciously high scores	target 0.86–0.92; audit any single feature scoring >0.85 alone
R5 abstention	naming a place on a flat distribution	district→cell→terminal gates in M3
R6 naive IST	timezone bugs	naive IST internally; <code>+05:30</code> only at JSON output

THE ONE-LINE PITCH

"We don't ask you to trust that we didn't cheat — we wrote tests that fail the build if we did, and they run on every change."

WHY R4 IS BACKWARDS ON PURPOSE

Most teams chase the highest accuracy. We *capped* ours at 86–92% and treat a single feature scoring above 0.85 as a leak to hunt. In this domain, a too-good number almost always means the model peeked at the answer.

6 · The data

FIELD GUIDE

Every row is synthetic, generated by `gen/`. This is a legal necessity — real money-mule records are personal financial data protected under the **DPDP Act 2023** and cannot go into an open prototype — and, done right, a strength.

The world

8,000 accounts (~6% mules in rings of 3), ~250,000 transactions, 600 complaints, ~700 ATMs across four Pune districts (Pune City, Pimpri-Chinchwad, Hinjawadi, Hadapsar), indexed with H3 hexagons (res-7 ≈ 5 km² cells). The generator is **deliberately hard**: mules and honest customers overlap heavily (six legitimate archetypes that look mule-like), some edges are unobserved, and labels carry noise — so the model learns real behaviour, not a watermark.

THE SEALED CORPUS — THE ANTI-CHEATING PROOF

The model is also tested on a *deliberately different* distribution it never trained on (different hotspots, slower forwarding, rarer fraud, a fraud type absent from training). Accuracy barely drops — **0.913** → **0.879**. A memoriser would collapse; ours generalises.

7 · The numbers & how to defend them

FIELD GUIDE

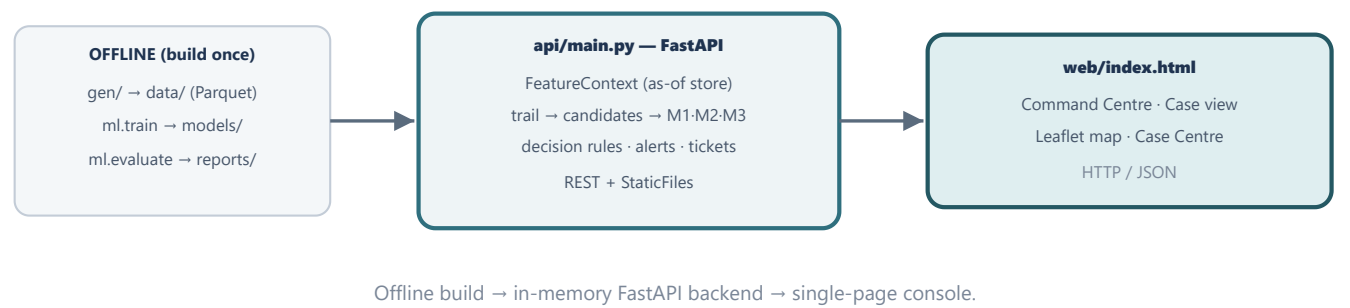
Number	Meaning	Defence
0.913	M1 ROC-AUC	honest, not inflated — capped target, audited features
0.879	on the sealed set	barely drops → generalises, not memorises
0.82	candidate recall@50	the true ATM is in our shortlist 82% of the time
~0.7 km	median error when we name a place	neighbourhood-accurate when we commit
50% → 91%	rules vs our model	the whole answer to "why not just rules?"
~12 min	median lead time when we can act	enough to freeze
~66%	already cashed out before filing	the honest case for faster reporting

Splits: chronological 0.913 · unseen-account 0.916 · sealed 0.879. **Baselines:** rule engine 0.500, logistic 0.840, most-frequent-ATM location 0.198 vs M3 0.207.

8 · Architecture & the technology stack

FIELD GUIDE

One Python backend, built from offline artefacts, serving one browser console over REST/JSON. No database server, no cloud, no keys — it runs from a folder of Parquet files and trained models.



Layer	Tools
Language	Python 3.12
Data	pandas, numpy, pyarrow (Parquet), h3 (spatial hex index)
Models	scikit-learn, xgboost (M1 & M3), networkx (graph), joblib
Service	FastAPI, uvicorn, pydantic (contracts), SQLite (tickets), StaticFiles
Console	vanilla HTML/CSS/JS, Leaflet.js (map), SVG overlay (flow lines)
Plots & tests	matplotlib, pytest (leakage suite), Make
Research track	PyTorch + PyTorch Geometric (temporal & hetero GNN experiments)

Repository at a glance

config.py	constants: districts, thresholds, tiers, horizons
contracts.py	Pydantic data contracts
gen/	synthetic world (params, atms, generator)
graphx/	trail, graph features, candidates, GNN research
ml/	feature store, train, predict, evaluate
api/main.py	FastAPI backend + decision rules + tickets
web/index.html	the single-page console
tests/	leakage guarantees

9 · Decision, alerts & the API

FIELD GUIDE

The models produce probabilities; plain, readable **rules** — deliberately not another model — turn them into an action an officer can audit.

Tier	Rule	Action
RED	$p_{10m} \geq 0.60$ AND tainted $\geq ₹5,000$	FREEZE now + alert
AMBER	$0.30 \leq p_{10m} < 0.60$	watch / email brief
GREY	below AMBER	keep tracing

Every recommendation carries a plain-English reason (the rule trace). Alerts route by district to the covering cyber police station (SHO), de-duplicated to one per account per 15 minutes. Alerts are *simulated* — payloads are built and logged, not actually sent.

The API (interactive docs at [/docs](#))

GET /queue	triage queue, sorted by imminence
GET /predict/{ack}	full forecast: cashout + channel + location + recommendation
GET /trail/{ack}	the reconstructed money flow
GET /hotspots	risk per H3 cell + ATM markers (date/amount/channel filters)
GET /explain/{ack}	human-readable forecast + rule trace
POST /alerts/dispatch · GET /alerts	build/log an alert; the delivery log
/tickets · /tickets/{ack} (+PATCH, /note, /resolve)	the Case Centre ticket lifecycle + CSV export

10 · The console

FIELD GUIDE

One HTML file, no build step, three views. A calm, mature command-console look (muted steel-teal, red = act, amber = watch).

- **Command Centre** — the overview: cases tracked, freeze-now count, money at risk, window left, metro risk map, and the urgency-sorted case queue.
- **Case view** — one case: the live money-flow map, the ranked target-ATM menu, the hazard/channel/location forecast, the freeze recommendation, and the cinematic replay.
- **Case Centre** — SOAR-style ticket lifecycle: priority, status, assignee, notes, resolution, audit trail, printable report and CSV export (backed by SQLite).

The live money-flow map

Leaflet with a dark Esri base + labels. Nodes are clean circular icons (victim, mules, ranked ATMs). Money is drawn as animated "waypoint" lines — gold for observed flow (victim→mules), red for the predicted routes (holding mule→each ATM). The lines are drawn in a **screen-space SVG overlay** glued to the map's pixel coordinates and trimmed to each circle's edge, so they stay pixel-perfect through zoom/pan and never bury the icons.

THE BEST DEMO MOMENT

Press *Replay simulation*: the case re-runs in fast motion inside a cinematic frame with a timecode and act labels. The gold flow traces first; the red target routes reveal at the cash-out climax — turning a probability into a visible, tense story.

11 · Run it & demo it

FIELD GUIDE

From zero

```
python -m venv .venv && .venv\Scripts\activate
pip install pandas numpy scikit-learn xgboost networkx \
    fastapi uvicorn pydantic pyarrow h3 matplotlib pytest

python -m gen.atms && python -m gen.generator          # build data
python -m gen.generator --sealed                       # sealed test set
python -m ml.train && python -m ml.evaluate             # train + evaluate
python -m pytest tests/ -q                             # leakage suite must pass

python demo.py                                         # headless story
python -m uvicorn api.main:app --port 8000            # open http://localhost:8000
```

Or: `make all && make test && make demo && make api`.

The 3-minute demo

1. Open the console → "officer's screen; queue sorted by urgency."
2. Click the top RED case → "traced to this account, holding ₹X."
3. Hazard curve → "high chance of cash-out in ten minutes."
4. Map + location → "this neighbourhood — and it *abstains* from naming one ATM. Honest AI."
5. Recommendation → "action is FREEZE, alert routes to this station."
6. Replay → "watch the money move and the risk rise."
7. Close → "91% vs 50% for rules — and it tells us when it doesn't know."

SAFETY NET

If the browser dies on stage, `python demo.py` prints the whole story to the terminal.

12 · Limitations & roadmap

FIELD GUIDE

Named plainly, because credibility comes from owning your weaknesses first.

Known limitation	The path forward
ATM coordinates are synthetic (around real centroids)	swap in real OpenStreetMap <code>amenity=atm</code> nodes — query already in <code>gen/atms.py</code>
All data is synthetic (DPDP Act 2023)	the generator is deliberately hard, so the lessons transfer; real deployment integrates with bank/complaint systems
Alerts are simulated (built & logged, not sent)	wire real SMS/email/webhook gateways and a real account-freeze API
Location ranker is the simple softmax version	add a pairwise ranker; the GNN research track (PyTorch Geometric, temporal GNN) is the documented accuracy upgrade
~66% of cases already cashed out before filing	the biggest win is upstream — bank-side auto-alerts that shrink the ~110-min reporting delay

NOT EVIDENCE

Outputs are decision-support for authorised investigators, not a licensed forecast of any real person or account.

13 · Glossary

FIELD GUIDE

ack_no	the 14-digit complaint id — a case's primary key
seed transaction	the one disputed payment the victim reported
mule / ring	a rented account moving stolen money; rings (size 3) share ATMs
trail / taint	the reconstructed money chain; the stolen portion of a balance
leaf / holding	an account still holding stolen funds — a freeze target
cash-out	the moment money becomes cash (ATM/POS) — the end of the trail
as-of	the prediction moment; features may read only earlier data
hazard curve	rising cash-out probability over the next 10–60 min (M1)
cell / terminal	a ~5 km ² H3 area / a single ATM
abstention	refusing to name one ATM when unsure; giving the cell instead
golden hour	the first 60 min after the fraud — best chance to freeze
lead time	minutes between the alarm and the cash-out (positive = early)
tier	RED = freeze now · AMBER = watch · GREY = keep tracing
sealed corpus	a different test dataset the model never trained on

SIH26184 · Predictive Analytics for Cybercrime Cash-out Forecasting · The Field Guide · Version 1.0. All data synthetic by design.
Decision-support for authorised investigators, not evidence. Read the Complete Manual for the full detail. Generated with Claude Code.